ControlFlow: Streaming Action Prediction via Dynamic Magnitude Scaling

Anonymous Author(s)
Affiliation
Address
email

Abstract: Robotic manipulation requires policies that can coordinate coarse long-range motion with fine-grained adjustment under real-time feedback constraints. Diffusion- and flow-based policies provide expressive action modeling, but trajectory-level generation can delay execution, while existing streaming flow policies do not explicitly adapt action-update magnitude across task phases. We introduce **ControlFlow**, an adaptive streaming flow policy that represents action generation as an action-space ODE and modulates the learned velocity field with an observation-conditioned magnitude variable. This modulation directly controls the effective scale of velocity updates, enabling phase-aware coarse-to-fine action generation without changing the streaming inference procedure. To improve velocity-field learning, we further propose multi-point conditional flow matching, which supervises multiple flow-time locations along each demonstration and provides denser training constraints without adding inference cost. Across visual, state-based, and language-conditioned manipulation benchmarks, ControlFlow improves rollout stability and task performance while preserving the low-latency streaming structure of action-space flow policies. Demo videos illustrating the learned behaviors and execution dynamics are available at https://skjefbekrngklrekg.github.io/Anonymous/test.html.

Keywords: Robotic Manipulation, Dynamic Action Magnitude, Flow Matching

1 Introduction

Robotic manipulation requires policies that coordinate coarse motion and precise adjustment under real-time feedback constraints. A robot may need to move rapidly toward an object, reduce action magnitude during contact, and perform fine-grained corrections near task completion. These action-scale changes reflect the phase structure of manipulation tasks, where target distance, contact state, object geometry, and task progress determine the appropriate control magnitude. Policies without explicit action-scale adaptation may overshoot during precision phases or move inefficiently during long-range phases.

Generative policies have become a powerful framework for visuomotor imitation learning. Diffusion policies generate action chunks through iterative denoising [1, 2], while flow-matching policies learn continuous vector fields for action generation [3, 4, 5]. However, many of these methods generate future action sequences before execution, coupling action prediction with a sampling process that can delay the first executable action and weaken real-time sensorimotor responsiveness.

Streaming Flow Policy (SFP) addresses this latency issue by progressively integrating a learned action-space velocity field and emitting actions during generation [6]. This formulation is well aligned with receding-horizon control, but it does not explicitly adapt the effective scale of velocity updates across task phases. As a result, scale variation must be absorbed implicitly by the learned velocity field when covering both large corrective motions and small terminal adjustments.

Submitted to the 10th Conference on Robot Learning (CoRL 2026). Do not distribute.

To address this challenge for precise manipulation, we propose **ControlFlow**, an adaptive streaming flow policy for phase-aware robot manipulation. ControlFlow retains the action-space streaming formulation, but introduces an observation-conditioned magnitude modulation variable λ to modulate the effective scale of the learned velocity field. During integration, λ modulates action-update magnitude rather than directly predicting actions, enabling larger updates during long-range motion and smaller updates during contact-rich or terminal adjustment phases. Furthermore, We introduce *multi-point conditional flow matching* for learning the action-space velocity field. Instead of supervising a single randomly sampled flow time per trajectory, our objective samples multiple flow-time points from the same demonstration and matches the corresponding trajectory-induced velocity targets. This provides denser supervision along each demonstrated action path, improving phase coverage and supporting more stable rollouts without changing the inference rule or adding inference-time cost.

Moreover, ControlFlow can be instantiated with different velocity backbones, including a U-Net model for controlled manipulation benchmarks and a DiT-style Transformer action expert with CLIP image-text conditioning for language-conditioned multi-task settings. A visual comparison of trajectory-level generation, streaming flow execution, and ControlFlow is provided in Appendix A. With extensive experiments on challenging tasks including Push-T, RoboMimic, and LIBERO, the results demonstrate that the adaptive magnitude modulation and multi-point supervision in our ControlFlow framework not only improve controlled manipulation performance but also maintain low streaming latency. In addition, we show that the same streaming formulation can be extended beyond the U-Net setting to competitive multi-task, language-conditioned manipulation.

Our contributions are summarized as follows:

- We introduce observation-conditioned magnitude modulation for streaming action-space flow policies, enabling the learned velocity field to adapt its effective action-update scale across manipulation phases.
- We propose multi-point conditional flow matching, which supervises multiple flow-time locations along each demonstration to improve velocity-field learning and rollout stability without modifying inference.

2 Related Work

Diffusion-, Flow-, and Streaming Policies. Generative policies are widely used for robot imitation learning, including diffusion policies that denoise future action trajectories [1, 7] and recent variants with transformer backbones, efficient sampling, and richer perceptual inputs [8, 2, 9]. Flow-matching policies instead learn continuous vector fields for action generation and have been applied to visuomotor manipulation, point-cloud-conditioned control, affordance-guided manipulation, and geometry-aware policy learning [3, 4, 5, 10, 11, 12]. However, trajectory-level generation often requires completing a sampling process before execution. Streaming formulations reduce this delay by dispatching actions during generation, including streaming diffusion policies [13] and Streaming Flow Policy, which integrates an action-space velocity field online [6]. ControlFlow follows this streaming formulation, but further introduces observation-conditioned magnitude modulation and multi-point flow-matching supervision to adapt action scale across task phases.

Adaptive and Multi-Scale Action Prediction. Recent methods study action prediction under temporal or scale heterogeneity. Diffusion–state-space policies model heterogeneous demonstration frequencies and modulate action step sizes for coarse-to-fine behavior [14]. Other approaches decompose actions into multi-scale components [15], use frequency-domain representations [16], or apply distillation and consistency training to reduce sampling cost [17, 18]. These works highlight the importance of motion-scale variation in robot demonstrations. ControlFlow addresses this issue by modulating the effective velocity magnitude of a streaming action-space ODE, without requiring a separate temporal hierarchy or explicit multi-granularity data.

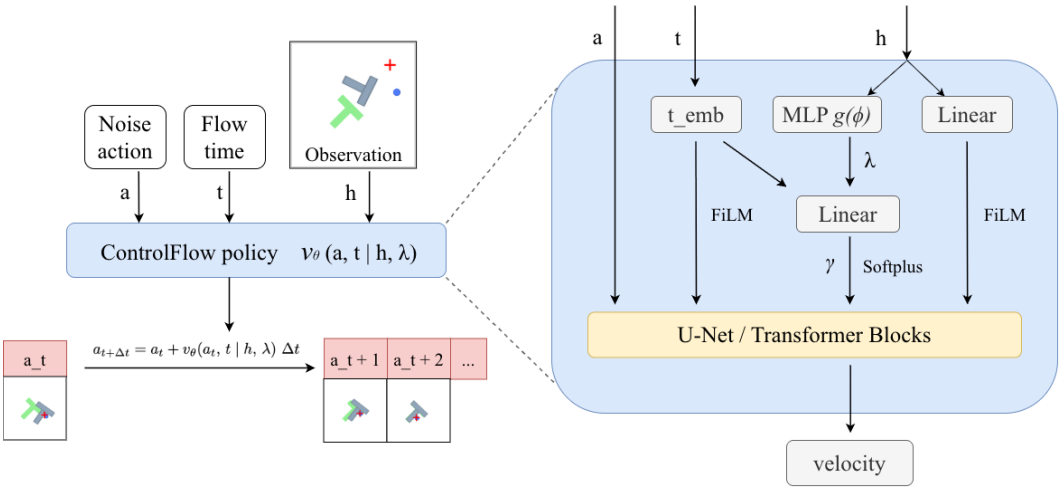


Figure 1: **ControlFlow overview.** ControlFlow predicts an action-space velocity field from the current action state, flow time, and observation history. The observation-conditioned modulation variable λ adjusts the effective magnitude of streaming velocity updates. In the U-Net version, λ is injected through FiLM-style residual scaling; in the DiT-style version, it is incorporated through token conditioning, AdaLN-style modulation, and explicit residual step scaling. During inference, actions are generated by integrating the predicted velocity field and can be streamed during generation.

3 Method

As illustrated in Fig. 1, ControlFlow formulates robotic policy learning as streaming integration of an action-space velocity field conditioned on the current action state, flow time, and observation history. The velocity model can be instantiated with either a conditional U-Net or a DiT-style Transformer action expert, while sharing the same streaming formulation, observation-conditioned magnitude modulation, and multi-point CFM objective. An observation-conditioned modulation variable λ modulates the effective magnitude of velocity updates, enabling phase-aware action scaling during streaming generation. Detailed architecture designs are provided in Appendix B.

3.1 ControlFlow

Following Streaming Flow Policy [6], ControlFlow represents a policy as an observation-conditioned velocity field over a continuous action space $\mathcal{A}$:

$$v_\theta(a, t \mid h, \lambda),$$

where $a \in \mathcal{A}$ is the current action state, $t \in [0, 1]$ is the normalized flow time, h is the observation history, and $\lambda = g_\phi(h)$ is the magnitude modulation variable. Given an initial action state, actions are generated by integrating

$$a(t) = a_0 + \int_0^t v_\theta(a(\tau), \tau \mid h, \lambda)\, d\tau.$$

Because integration proceeds sequentially, actions can be streamed during generation without waiting for the full trajectory.

Trajectory Velocity Definition. Given demonstrations $\mathcal{D} = \{(h_i, \xi_i)\}_{i=1}^M$ with action trajectories $\xi_i : [0, T_{\text{pred}}] \to \mathcal{A}$, each trajectory is rescaled to $t \in [0, 1]$. Following Streaming Flow Policy [6], we construct the trajectory-induced target velocity field

$$v_\xi(a, t) = \dot{\xi}(t) - k(a - \xi(t)),$$

where $\dot{\xi}(t)$ is the trajectory velocity and $k > 0$ is a stabilizing gain that attracts the current action state toward the demonstrated path. The conditional distribution around a demonstration at flow time

t is denoted by $p_\xi(a \mid t)$, and samples from this distribution expose the model to local deviations during velocity matching.

Observation-Conditioned Magnitude Modulation. To adapt action scale during streaming execution, ControlFlow predicts a positive observation-conditioned modulation variable

$$\lambda = g_\phi(h), \qquad g_\phi : \mathbb{R}^d \to \mathbb{R}_+.$$

The scalar λ represents the effective action-update scale implied by the current observation and task phase. It does not directly predict actions; instead, it modulates the learned velocity field.

For U-Net-based velocity models, λ is embedded and combined with the flow-time embedding to produce a positive residual scaling coefficient:

$$\gamma = \mathrm{Softplus}(W_m[\, t_{\mathrm{emb}}, f_\lambda \,]).$$

This coefficient is injected into each conditional residual block:

$$h_{\ell+1} = h_\ell + \gamma F_\theta(h_\ell).$$

The positivity constraint makes λ act as a magnitude controller rather than a direction-reversing signal.

For DiT-style velocity models, λ is used both as a conditioning token and as an explicit residual step scale. The action state forms the query, while visual, language, and state tokens provide cross-attention memory. In addition to AdaLN-style shift-scale modulation and residual gates, ControlFlow-T computes

$$\delta = \lambda \cdot \mathrm{Softplus}\left(\mathrm{MLP}_\delta([t_{\mathrm{emb}}, f_\lambda])\right),$$

and applies it to Transformer residual updates:

$$h_{\ell+1} = h_\ell + \delta \cdot \mathcal{F}_\theta(h_\ell; h, t, \lambda),$$

where $\mathcal{F}_\theta$ denotes the gated self-attention, cross-attention, and MLP transformations.

Both U-Net and DiT-style instantiations share the same streaming velocity-field interface, and the same action-space integration rule. Thus, λ enables phase-aware action scaling without modifying inference.

Multi-Point Conditional Flow Matching. We train the policy velocity field using conditional flow matching (CFM) [19, 3]. Given a demonstration dataset $\mathcal{D} = \{(h, \xi)\} \sim p_{\mathcal{D}}$, the objective is to match the learned velocity field to the trajectory-induced target velocity $v_\xi(a, t)$.

Instead of sampling a single flow time per trajectory, MP-CFM samples N flow-time points from the same demonstrated action path:

$$t_1, \ldots, t_N \sim \mathcal{U}[0, 1].$$

For each sampled time t_j, we sample $a_j \sim p_\xi(a \mid t_j)$ and supervise the corresponding target velocity:

$$\mathcal{L}_{\text{MP-CFM}}(v_\theta, p_{\mathcal{D}}) = \mathbb{E}_{(h,\xi)\sim p_{\mathcal{D}}} \mathbb{E}_{\{t_j\}_{j=1}^N} \left[\frac{1}{N} \sum_{j=1}^{N} \frac{1}{2} \left\| v_\theta(a_j, t_j \mid h, \lambda) - v_\xi(a_j, t_j) \right\|_2^2 \right], \quad (1)$$
$$a_j \sim p_\xi(a \mid t_j), \qquad \lambda = g_\phi(h).$$

When $N = 1$, Eq. 1 reduces to standard single-point CFM. For $N > 1$, MP-CFM provides denser within-trajectory supervision along the integration interval. This is different from simply increasing the number of independent trajectories in a minibatch: the sampled points share the same observation context h and the same predicted modulation variable λ, while their action states and target velocities vary across flow time. Thus, MP-CFM exposes the velocity model and the modulation pathway to phase-dependent velocity targets along the same demonstrated path.

MP-CFM only changes training-time supervision. It does not modify the velocity-model architecture, streaming integration rule, or inference-time latency. Further discussion on rollout stability and the learning signal for λ is provided in Appendix E.

Inference. At inference time, ControlFlow is executed in a receding-horizon manner. Although the policy is trained with prediction horizon T_{pred}, it executes only a shorter chunk horizon $T_{\text{chunk}} \leq T_{\text{pred}}$ at each prediction cycle.

Given the current observation history h, the initial action state is initialized from the most recently executed action. Actions are generated by integrating the learned velocity field over normalized flow time $t \in [0, T_{\text{chunk}}/T_{\text{pred}}]$:

$$a_{t+\Delta t} = a_t + v_\theta(a_t, t \mid h, \lambda)\Delta t,$$

where Δt is the integration step size and $\lambda = g_\phi(h)$ is the observation-conditioned magnitude modulation variable.

Within each generated chunk, the observation history h is fixed, and actions are streamed as they are produced. Across chunks, h is updated with new observations, allowing receding-horizon feedback. Thus, ControlFlow preserves the streaming inference structure while adaptively modulating action magnitude through the learned velocity field.

How λ Learns Action Magnitude. The modulation variable λ is learned implicitly through the MP-CFM objective, without explicit motion-scale labels. Its learning signal comes from matching trajectory-induced target velocities whose magnitudes vary across task phases.

All demonstrations are represented on a normalized flow-time domain $t \in [0, 1]$. For a trajectory originally defined over horizon T, the rescaling $t' = t/T$ gives

$$\frac{d\xi}{dt'} = T \cdot \frac{d\xi}{dt}.$$

Thus, trajectory segments with larger displacement over the normalized interval induce larger target velocity magnitudes, while slower alignment or contact-rich segments induce smaller ones.

The target velocity magnitude also depends on the stabilizing correction toward the demonstrated path. Since $\lambda = g_\phi(h)$ modulates the learned velocity field, the velocity-matching loss backpropagates through $v_\theta(a, t \mid h, \lambda)$ to update the modulation predictor. MP-CFM further exposes the same λ to multiple flow-time locations from one demonstrated path, aggregating phase-dependent velocity-magnitude signals within each update. Consequently, λ acts as an observation-conditioned modulation factor for coarse-to-fine streaming action generation. A more detailed gradient and rollout-stability analysis is provided in Appendix E.

4 Experiments

We evaluate ControlFlow through two benchmark settings and one targeted rollout-efficiency analysis. First, we use Push-T and RoboMimic to conduct controlled evaluations under state-based and image-based observations. These experiments use the U-Net-based velocity model and focus on the effects of adaptive magnitude modulation and multi-point CFM supervision. Second, we evaluate ControlFlow-T on LIBERO to study whether the same streaming flow formulation can be extended to language-conditioned multi-task manipulation with a DiT-style action expert and CLIP image-text conditioning. Finally, we further analyze efficiency and fixed-λ variants against the closest Multi-task DiT baseline, measuring success rate, completion steps, and amortized per-step policy latency.

We refer the readers for additional architecture details in Appendix B, training configurations in Appendix D, benchmark descriptions in Appendix F, MP-CFM analysis in Appendix E, adaptive λ analysis in Appendix G, action-horizon sensitivity in Appendix H, and efficiency results in Appendix C.

4.1 Evaluation Settings and Implementation Details

Datasets. We evaluate ControlFlow on Push-T [20], five RoboMimic tasks [21] (Can, Lift, Square, ToolHang, and Transport), and LIBERO [22]. Push-T and RoboMimic provide controlled state- and image-based manipulation settings, covering contact-rich pushing, precision placement, and

Table 1: State-based Push-T and RoboMimic results with average/maximum scores and per-step policy latency. For ControlFlow, N denotes the number of MP-CFM supervision points and only affects training.

Policy	Push-T		RoboMimic					
	Score	Lat(ms).	Can	Lift	Square	ToolHang	Transport	Lat(ms).
DP (DDPM 100)	0.91/0.92	41.1	0.95/0.98	**1.00/1.00**	**0.93/0.96**	0.35/0.62	0.82/0.93	54.2
DP (DDIM 10)	0.87/0.89	4.3	0.94/0.98	**1.00/1.00**	0.91/0.94	0.40/0.65	0.84/0.91	5.8
Streaming DP	0.86/0.88	27.2	0.99/**1.00**	0.97/**1.00**	0.87/0.94	0.42/0.65	0.83/0.92	32.4
SFP	0.92/0.93	3.5	0.98/**1.00**	**1.00/1.00**	0.78/0.94	0.53/0.70	0.83/0.93	4.5
ControlFlow (N=1)	0.94/**1.00**	**3.2**	**1.00/1.00**	**1.00/1.00**	**0.93**/0.95	0.57/0.72	0.86/0.94	**4.3**
ControlFlow (N=4)	0.94/**1.00**	**3.2**	**1.00/1.00**	**1.00/1.00**	**0.93**/0.95	**0.61**/0.72	0.86/0.95	**4.3**
ControlFlow (N=8)	**0.95/1.00**	3.3	**1.00/1.00**	**1.00/1.00**	**0.93/0.96**	**0.61/0.73**	**0.90/0.96**	4.6

Table 2: Image-based Push-T and RoboMimic results with average/maximum scores and per-step policy latency. All ControlFlow variants share the same inference procedure.

Policy	Push-T		RoboMimic					
	Score	Lat(ms).	Can	Lift	Square	ToolHang	Transport	Lat(ms).
DP (DDPM 100)	0.89/0.92	127.0	0.98/**1.00**	**1.00/1.00**	0.93/0.96	0.82/0.84	0.89/0.93	138.6
DP (DDIM 10)	0.82/0.85	11.2	0.97/0.98	0.99/**1.00**	0.91/0.95	0.86/0.88	0.90/0.94	11.5
Streaming DP	0.78/0.80	76.8	0.98/0.98	0.99/**1.00**	**0.94**/0.96	0.81/0.85	0.89/0.91	82.1
SFP	0.84/0.85	8.8	0.98/**1.00**	0.99/**1.00**	0.78/0.84	0.84/0.87	0.92/0.94	9.4
ControlFlow (N=1)	0.87/**1.00**	**4.2**	0.99/**1.00**	**1.00/1.00**	0.92/0.95	0.84/0.89	0.92/0.96	**4.8**
ControlFlow (N=4)	0.89/**1.00**	**4.2**	**1.00/1.00**	**1.00/1.00**	0.93/0.95	**0.87**/0.90	0.92/**0.97**	**4.8**
ControlFlow (N=8)	**0.93/1.00**	4.4	**1.00/1.00**	**1.00/1.00**	**0.94/0.97**	**0.87/0.92**	**0.93/0.97**	5.0

longer-horizon transport. LIBERO evaluates language-conditioned multi-task manipulation across the Spatial, Object, Goal, and Long suites. Environment details are provided in Appendix F.

Evaluation Protocol. For Push-T and RoboMimic, we compare against diffusion, streaming diffusion, and streaming flow baselines [1, 13, 6]. Diffusion policies are evaluated with DDPM sampling and accelerated DDIM sampling [23, 24]. We evaluate the last 10 checkpoints and roll out 100 randomized episodes per checkpoint, reporting average and best-checkpoint scores. For LIBERO, we follow prior language-conditioned manipulation protocols [25] and compare against diffusion, streaming-flow, Multi-task DiT, and VLA baselines. Additional latency details are provided in Appendix C.

Training. Push-T and RoboMimic use the U-Net-based ControlFlow model, while LIBERO uses ControlFlow-T with a DiT-style Transformer action expert and CLIP image-text conditioning. We evaluate $N \in \{1, 4, 8\}$ MP-CFM supervision points, where $N = 1$ recovers standard single-time CFM. Larger N only changes training-time supervision and leaves inference unchanged. Architecture and training details are provided in Appendix B and Appendix D.

4.2 Results on Controlled Manipulation Benchmarks

As shown in Tables 1 and 2, ControlFlow achieves competitive or improved performance on Push-T and RoboMimic while maintaining low per-step inference latency. These experiments use the U-Net-based velocity model as a controlled setting for evaluating adaptive magnitude modulation and MP-CFM before extending to language-conditioned multi-task manipulation.

On Push-T, ControlFlow consistently reaches the maximum score. Since the overlap-based metric depends on precise final alignment with the target region, this task requires coarse pushing followed by fine terminal adjustment, making it sensitive to adaptive action scaling. On RoboMimic, ControlFlow maintains strong performance on easier tasks such as Can and Lift, while showing clearer gains on longer-horizon, precision, or contact-sensitive tasks such as Square, ToolHang, and Transport. The image-based results follow the same trend, indicating improved rollout stability with efficient online execution.

Table 3: LIBERO multi-task evaluation. We report success rates(%) on the Spatial, Object, Goal, and Long suites. ControlFlow-U uses the U-Net velocity model without language conditioning, while ControlFlow-T uses a Transformer action expert with CLIP image-text conditioning.

Policy	Backbone / Language Cond.	Spatial	Object	Goal	Long	Avg.
Diffusion Policy (72M) [25]	U-Net / No	78.3	92.5	68.3	50.5	72.4
Streaming Flow Policy (72M)	U-Net / No	79.5	93.3	72.4	48.7	73.5
Multi-task DiT (0.45B) [26]	DiT / Yes	87.0	**98.2**	93.8	83.2	90.6
SmolVLA (0.24B) [25]	VLA / Yes	87.0	93.0	88.0	63.0	82.8
SmolVLA (2.25B) [25]	VLA / Yes	93.0	94.0	91.0	77.0	88.8
OpenVLA (7B) [27]	VLA / Yes	84.7	88.4	79.2	53.7	76.5
OpenVLA-OFT (7B) [28]	VLA / Yes	**95.2**	94.2	**95.2**	**93.2**	**94.5**
ControlFlow-U (N=8) (73M)	U-Net / No	81.5	93.6	78.2	52.1	76.3
ControlFlow-T (N=8) (0.5B)	DiT-style / Yes	94.3	96.2	95.1	85.7	92.8

Increasing the number of MP-CFM supervision points generally improves performance by providing denser velocity-field supervision along each demonstration. Moving from $N = 1$ to $N = 4$ improves phase coverage, while $N = 8$ brings further gains on harder tasks but begins to saturate. Unlike simply increasing minibatch size, MP-CFM ties multiple flow-time targets to the same observation context and predicted λ, exposing the velocity model to phase-dependent target velocities along a single demonstrated path. Since N only affects training-time supervision, all variants share the same streaming integration rule and comparable inference latency.

We further isolate adaptive magnitude modulation in Appendix G. Fixed λ values lead to performance sensitivity across rollout phases, whereas adaptive λ achieves stronger Push-T image-based performance, suggesting that a single global action scale is insufficient for both coarse motion and terminal adjustment. Additional analyses of adaptive λ behavior and action-horizon sensitivity are provided in Appendix G and Appendix H.

4.3 Results on Language-Conditioned Multi-Task Manipulation

Table 3 reports LIBERO success rates across the Spatial, Object, Goal, and Long suites. This setting evaluates whether ControlFlow can be extended from controlled manipulation benchmarks to language-conditioned multi-task action generation. Unlike the Push-T and RoboMimic experiments, which use the U-Net-based velocity model, LIBERO uses ControlFlow-T, a DiT-style Transformer action expert with CLIP image-text conditioning and the same streaming flow formulation.

The results show that language-conditioned Transformer and VLA-style policies substantially outperform non-language-conditioned U-Net baselines on LIBERO. ControlFlow-U improves over diffusion and streaming-flow baselines under the same non-language-conditioned setting, but remains limited by the lightweight U-Net backbone and the absence of language conditioning. ControlFlow-T benefits from CLIP visual-language context and a DiT-style action expert, leading to stronger performance across the LIBERO suites.

Despite using a smaller model than large-scale VLA systems, ControlFlow-T achieves competitive multi-task performance. With a 0.5B DiT-style backbone, ControlFlow-T outperforms OpenVLA and SmolVLA variants on average and approaches the stronger fine-tuned OpenVLA-OFT baseline. This suggests that a moderate-scale streaming action expert can provide strong language-conditioned manipulation performance without relying on a very large VLA backbone.

Overall, the LIBERO results demonstrate the extensibility of ControlFlow beyond the U-Net setting. The same action-space streaming formulation can be instantiated with a DiT-style action expert and CLIP image-text conditioning for multi-task manipulation. The specific effects of adaptive magnitude modulation are analyzed in the controlled Push-T and RoboMimic experiments, while LIBERO focuses on scalability to language-conditioned policy learning.

Table 4: LIBERO-Spatial comparison against the closest DiT-style baseline and fixed-λ variants. Each task is evaluated with 20 episodes under the official 280-step horizon. We report success rate, average completion steps over successful episodes, and amortized per-environment-step policy latency.

Policy	**SR (%)** $\uparrow$	**Succ. Steps** $\downarrow$	**Lat. (ms)** $\downarrow$
Multi-task DiT [26]	87.0	126.2	40.63
ControlFlow-T, fixed $\lambda = 0.5$	92.6	106.2	**9.67**
ControlFlow-T, fixed $\lambda = 1.0$	92.4	97.8	**9.67**
ControlFlow-T, fixed $\lambda = 1.5$	85.2	**90.2**	**9.67**
ControlFlow-T, dynamic $\lambda \in [0.5, 1.5]$	**94.3**	92.5	**9.67**

4.4 LIBERO-Spatial Rollout Efficiency and Lambda Ablation

Since ControlFlow-T uses a DiT-style action expert rather than a large VLA backbone, the closest comparison is the Multi-task DiT baseline. We therefore further evaluate rollout efficiency and fixed-λ variants on the LIBERO-Spatial suite. All policies are evaluated with 20 episodes per task under the official 280-step episode horizon. Table 4 reports success rate, average completion steps over successful episodes, and amortized per-environment-step policy latency.

All ControlFlow-T variants are trained on the full LIBERO dataset with the same batch size and evaluated at 80k training steps, while the Multi-task DiT baseline is trained for 100k steps. ControlFlow-T already surpasses the 100k-step Multi-task DiT baseline at its 60k checkpoint, suggesting faster convergence under the same data setting. As shown in Table 4, ControlFlow-T with dynamic λ achieves the highest success rate while completing successful episodes in 92.5 executed steps on average, compared with 126.2 steps for Multi-task DiT. It also reduces amortized per-step policy latency from 40.63 ms to 9.67 ms. Additional latency-efficiency analyses are provided in Appendix C.

The fixed-λ variants further clarify the role of adaptive magnitude modulation. A larger fixed value, $\lambda = 1.5$, yields the shortest completion length among successful episodes, but its success rate drops to 85.2%, indicating that large action updates can accelerate successful rollouts while increasing failure risk. A smaller fixed value, $\lambda = 0.5$, achieves a success rate close to the dynamic variant, suggesting that many LIBERO-Spatial tasks favor smaller and more precise action updates. However, a single fixed action scale remains suboptimal overall: dynamic λ provides the best balance between robust completion and efficient rollout length. This complements the Push-T analysis in Appendix G, where a broader range of action magnitudes is required and adaptive λ is more important for switching between coarse motion and terminal adjustment.

5 Conclusion

We introduced ControlFlow, an adaptive streaming flow policy that uses an observation-conditioned modulation variable λ to adjust the effective magnitude of an action-space velocity field. This links task context to action-update scale, enabling coarse-to-fine behavior within the same streaming generation process while preserving low-latency online execution. We further proposed multi-point conditional flow matching, which supervises multiple flow-time locations along each demonstration to improve velocity-field learning without modifying inference. Experiments on Push-T and RoboMimic show improved controlled manipulation performance across state-based and image-based settings. Fixed- and dynamic-λ analyses indicate that a single global action scale is suboptimal: larger fixed values can complete successful rollouts faster but increase failure risk, while dynamic λ better balances success rate, rollout efficiency, and terminal adjustment. On LIBERO, ControlFlow-T extends the same formulation to a DiT-style CLIP-conditioned action expert and outperforms the closest Multi-task DiT baseline in success rate, completion steps, and amortized per-step policy latency. Overall, ControlFlow provides a practical mechanism for adaptive action scaling in streaming flow policies and offers a compatible path toward larger-scale Vision-Language-Action policies.

References

[1] C. Chi, S. Feng, Y. Du, J. Xu, S. Cousins, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems (RSS)*, 2023.

[2] X. Liu, K. Y. Ma, C. Gao, and M. Z. Shou. Diffusion models in robotics: A survey. *arXiv preprint*, 2025.

[3] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.

[4] E. Chisari, N. Heppert, M. Argus, T. Welschehold, T. Brox, and A. Valada. Learning robotic manipulation policies from point clouds with conditional flow matching. In *Conference on Robot Learning (CoRL)*, 2025.

[5] F. Zhang and M. Gienger. Affordance-based robot manipulation with flow matching. *arXiv preprint arXiv:2409.01083*, 2024.

[6] S. Jiang, X. Fang, N. Roy, T. Lozano-Pérez, L. P. Kaelbling, and S. Ancha. Streaming flow policy: Simplifying diffusion/flow-matching policies by treating action trajectories as flow trajectories. In *Conference on Robot Learning (CoRL)*, 2025.

[7] M. Janner, Y. Du, J. B. Tenenbaum, and S. Levine. Planning with diffusion for flexible behavior synthesis. In *International Conference on Machine Learning (ICML)*, 2022.

[8] M. Reuss, J. Pari, P. Agrawal, and R. Lioutikov. Efficient diffusion transformer policies with mixture of denoising experts for multitask learning. In *International Conference on Learning Representations (ICLR)*, 2025.

[9] Y. Ze et al. 3d diffusion policy: Generalizable visuomotor policy learning via 3d representations. *arXiv preprint arXiv:2403.03954*, 2024.

[10] M. Braun, N. Jaquier, L. Rozo, and T. Asfour. Riemannian flow matching policy for robot motion learning. *arXiv preprint arXiv:2403.10672*, 2024.

[11] S. Wang, L. Wang, S. Zhou, J. Tian, J. Li, H. Sun, and W. Tang. Flowram: Grounding flow matching policy with region-aware mamba framework for robotic manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[12] R. Xue, J. Mao, M. Zhang, and Y. Wang. Recflow policy: Fast and accurate visuomotor policy learning via rectified action flow. In *ICLR Workshop on Robot Learning*, 2025.

[13] H. Liu, Y. Chen, and P. Agrawal. Streaming diffusion policy for real-time robot control. In *Conference on Robot Learning*, 2023.

[14] N. Oh, J. Jang, M. Jung, and D. Park. Diffusion-ssm based policy learning for multi-granularity action prediction. In *Human to Robot (H2R) Workshop at Conference on Robot Learning (CoRL)*, 2025.

[15] Z. Gong, P. Ding, S. Lyu, S. Huang, W. Sun, Z. Zhao, D. Fan, and D. Wang. Carp: Visuomotor policy learning via coarse-to-fine autoregressive prediction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025.

[16] C. Yang, Y. Dong, G. Tian, H. Ge, and H. Zhu. Wavelet policy: Imitation policy learning in frequency domain with wavelet transforms. *arXiv preprint arXiv:2504.04991*, 2025.

[17] B. Jia, P. Ding, C. Cui, M. Sun, P. Qian, S. Huang, Z. Fan, and D. Wang. Score and distribution matching policy: Advanced accelerated visuomotor policies via matched distillation. *arXiv preprint*, 2024. arXiv:2412.09265.

[18] Y. Su, N. Liu, D. Chen, Z. Zhao, K. Wu, M. Li, Z. Xu, Z. Che, and J. Tang. Freqpolicy: Efficient flow-based visuomotor policy via frequency consistency. *arXiv preprint arXiv:2506.08822*, 2025.

[19] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[20] P. Florence, C. Lynch, A. Zeng, O. A. Ramirez, A. Wahid, L. Downs, A. Wong, J. Lee, I. Mordatch, and J. Tompson. Implicit behavioral cloning. In *Proceedings of the 5th Conference on Robot Learning*, volume 164 of *Proceedings of Machine Learning Research*, pages 158–168. PMLR, 2022.

[21] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. In *Conference on Robot Learning*, volume 155 of *Proceedings of Machine Learning Research*, pages 545–560. PMLR, 2021.

[22] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.

[23] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851, 2020.

[24] J. Song, C. Meng, and S. Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations (ICLR)*, 2021.

[25] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.

[26] B. Jones. Dissecting and open-sourcing multitask diffusion transformer policy, 2025. URL https://brysonkjones.substack.com/p/dissecting-and-open-sourcing-multitask-diffusion-transformer-policy. Blog post.

[27] M. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[28] M. J. Kim, C. Finn, and P. Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.

[29] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville. Film: Visual reasoning with a general conditioning layer. *arXiv preprint arXiv:1709.07871*, 2017.

[30] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[31] E. Todorov, T. Erez, and Y. Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, 2012. doi:10.1109/IROS.2012.6386109.

[32] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z.

Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

[33] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, J. Jang, Z. Jiang, J. Kautz, K. Kundalia, L. Lao, Z. Li, Z. Lin, K. Lin, G. Liu, E. Llontop, L. Magne, A. Mandlekar, A. Narayan, S. Nasiriany, S. Reed, Y. L. Tan, G. Wang, Z. Wang, J. Wang, Q. Wang, J. Xiang, Y. Xie, Y. Xu, Z. Xu, S. Ye, Z. Yu, A. Zhang, H. Zhang, Y. Zhao, R. Zheng, and Y. Zhu. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.

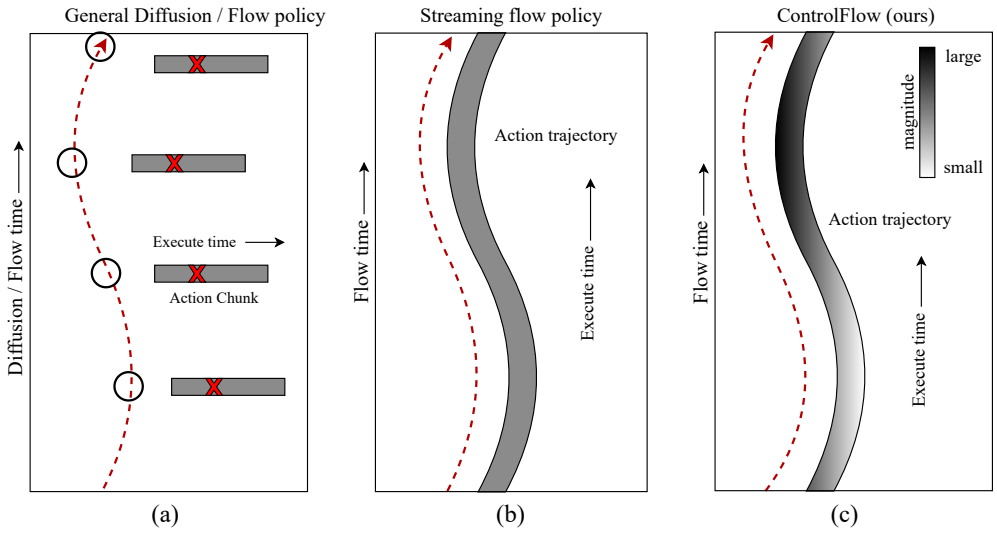


Figure 2: **Action generation and execution paradigms in generative robotic policies. (a)** Diffusion- and flow-based policies [1, 5] typically generate fixed-length action trajectories or chunks before execution, which provides expressive sequence modeling but delays the first executable action. **(b)** Streaming Flow Policy [6] progressively integrates a learned action-space velocity field and streams actions during generation, but does not explicitly adapt the effective action magnitude across task phases. **(c) ControlFlow** extends streaming action generation with observation-conditioned magnitude modulation and multi-point flow-matching supervision, enabling phase-aware coarse-to-fine control while preserving the streaming inference structure.

Appendix

A Action Generation Paradigms

This section provides a visual comparison of different action generation and execution paradigms in generative robotic policies. The key distinction lies in whether action generation must be completed before execution and whether the effective action magnitude can be adapted during execution. Trajectory-level diffusion and flow policies generate a full future action sequence before execution, while streaming flow policies progressively integrate an action-space velocity field and can emit actions online. ControlFlow follows the streaming formulation but further introduces observation-conditioned magnitude modulation, allowing the velocity field to adapt its effective update scale across different task phases.

Compared with trajectory-level generation, streaming execution reduces the delay before the first action can be executed. Compared with standard Streaming Flow Policy, ControlFlow additionally modulates the magnitude of the velocity field using the observation-conditioned variable λ. This allows the same streaming integration process to support larger updates during coarse motion and smaller updates during precise adjustment, without changing the inference-time integration rule.

B Network Architecture

ControlFlow parameterizes an action-space velocity field

$$v_\theta(a, t \mid h, \lambda),$$

where a is the current action state, t is the normalized flow time, h is the observation context, and λ is the observation-conditioned magnitude modulation variable. The velocity field outputs a vector

with the same dimensionality as the action, which is integrated online to generate executable actions in a streaming manner.

ControlFlow supports two velocity-model instantiations. For controlled Push-T and RoboMimic experiments, we use a lightweight U-Net-based velocity model to isolate the effects of adaptive magnitude modulation and multi-point CFM. For LIBERO, we use a DiT-style Transformer action expert with CLIP image-text conditioning to evaluate language-conditioned multi-task manipulation. Both instantiations share the same interface $v_\theta(a, t \mid h, \lambda)$ and the same streaming integration rule.

B.1 U-Net Velocity Model

For the U-Net version, the current action state a_t is treated as a temporal signal and serves as the primary input to a conditional U-Net. The model is conditioned on the normalized flow time $t \in [0, 1]$ and the observation context h through feature-wise affine modulation (FiLM) [29]. The flow-time embedding and the magnitude modulation embedding are combined to produce residual scaling coefficients, which are injected into each conditional residual block.

Given a residual transformation F_θ with input feature h_ℓ, the modulated block update is

$$h_{\ell+1} = h_\ell + \gamma F_\theta(h_\ell),$$

where γ is obtained from the joint embedding of flow time and λ. This design allows the U-Net to adjust the effective magnitude of the predicted velocity field while preserving the same action-space integration rule.

For image-based observations in the U-Net setting, RGB inputs are encoded using a ResNet-18 [30] backbone to produce a compact observation representation. This representation serves as the global conditioning signal h and is further processed by a lightweight predictor to produce the positive modulation variable λ. The resulting architecture is used for controlled comparisons where the goal is to isolate the effect of adaptive magnitude modulation and multi-point CFM supervision.

B.2 DiT-style Velocity Model with CLIP Token Conditioning

For multi-task and language-conditioned manipulation, we instantiate ControlFlow with a DiT-style Transformer velocity expert. This variant, denoted as ControlFlow-T, is used for LIBERO experiments and preserves the same streaming action-space velocity interface as the U-Net version:

$$v_\theta(a, t \mid h, \lambda).$$

Unlike standard DiT policies that directly generate an action chunk, ControlFlow-T predicts an action-space velocity field that is integrated online during streaming execution. Thus, the Transformer expert acts as a velocity model rather than a direct action-sequence decoder.

CLIP visual and language conditioning. Given visual observations, ControlFlow-T uses a CLIP vision encoder to extract both pooled visual features and compressed patch-level visual tokens. The pooled visual features are used to form the global observation context, while the patch tokens are retained as token-level cross-attention memory. For language-conditioned tasks, task instructions are encoded with a CLIP text encoder, which produces both pooled language features and token-level language embeddings. Robot state observations, when available, are projected into state tokens through a lightweight linear layer.

The visual, language, and state tokens are concatenated into a token-level context memory:

$$M_{\text{ctx}} = [M_{\text{img}}, M_{\text{text}}, M_{\text{state}}],$$

with a corresponding attention mask. In parallel, pooled visual features, pooled language features, and proprioceptive features are concatenated into a global observation context. This context is used for magnitude prediction and global Transformer conditioning, while M_{ctx} provides fine-grained visual-language-state information through cross-attention.

Action query and global conditioning. The current action state a, normalized flow time t, and modulation variable λ are embedded separately:

$$z_a = W_a a, \qquad z_t = \mathrm{MLP}_t(\mathrm{SinEmb}(t)), \qquad z_\lambda = \mathrm{MLP}_\lambda(\mathrm{SinEmb}(\lambda)).$$

These embeddings are fused to form the action query:

$$z_q = \mathrm{MLP}_q\left([z_a, z_t, z_\lambda]\right).$$

The global observation context is projected into a pooled context token:

$$z_h = \mathrm{MLP}_h(h).$$

We append the pooled context token, flow-time token, and magnitude token to the token-level memory:

$$M = [M_{\mathrm{ctx}}, z_h, z_t, z_\lambda].$$

Thus, the Transformer velocity expert receives both fine-grained token-level context and compact global conditioning information.

AdaLN-Zero modulation. Each Transformer block contains self-attention, cross-attention, and MLP branches. A conditioning vector is computed from the pooled context token, flow-time token, and magnitude token:

$$c = \mathrm{MLP}_c\left([z_h, z_t, z_\lambda]\right).$$

This conditioning vector generates shift, scale, and gate parameters for the three branches:

$$(\Delta_s, \alpha_s, g_s, \Delta_c, \alpha_c, g_c, \Delta_m, \alpha_m, g_m) = \mathrm{MLP}_{\mathrm{AdaLN}}(c).$$

For a normalized hidden state x, the FiLM-style modulation is

$$\mathrm{Mod}(x; \Delta, \alpha) = x \odot (1 + \alpha) + \Delta.$$

The AdaLN modulation head is zero-initialized, so the residual branches start with near-zero contribution and are gradually learned during training.

Frequency-scaled residual velocity updates. Beyond using λ as a conditioning token, ControlFlow-T explicitly uses it to scale Transformer residual updates. A positive learned scale is computed from the flow-time and magnitude embeddings:

$$s_\delta = \mathrm{Softplus}\left(\mathrm{MLP}_\delta\left([z_t, z_\lambda]\right)\right),$$

and the residual step-scaling factor is defined as

$$\delta = \lambda \cdot s_\delta.$$

This makes λ an explicit multiplicative factor in the velocity-field computation, rather than only a generic conditioning signal.

With this factor, each Transformer block is updated as

$$x \leftarrow x + \delta\, g_s \cdot \mathrm{SA}\left(\mathrm{Mod}(\mathrm{LN}(x); \Delta_s, \alpha_s)\right),$$

$$x \leftarrow x + \delta\, g_c \cdot \mathrm{CA}\left(\mathrm{Mod}(\mathrm{LN}(x); \Delta_c, \alpha_c), M\right),$$

$$x \leftarrow x + \delta\, g_m \cdot \mathrm{MLP}\left(\mathrm{Mod}(\mathrm{LN}(x); \Delta_m, \alpha_m)\right).$$

Here, SA, CA, and MLP denote the self-attention, cross-attention, and feed-forward branches. The gates g_s, g_c, and g_m are produced by AdaLN conditioning, while δ provides an additional magnitude-dependent residual scale shared across branches. This structure makes the predicted modulation variable directly affect the scale of velocity-field computation inside the Transformer.

After all Transformer blocks, the hidden action token is normalized and projected back to the action dimension:

$$v_\theta(a, t \mid h, \lambda) = W_o \mathrm{LN}(z_v).$$

The resulting vector is the predicted action-space velocity used by the streaming integrator.

B.3 Magnitude Predictor

ControlFlow predicts an observation-conditioned modulation variable from the global observation context:

$$\lambda = g_\phi(h).$$

In ControlFlow-T, g_ϕ is implemented as a lightweight frequency-like predictor initialized around a usable default value. The predictor outputs a scalar modulation value that represents the action-update scale implied by the current observation and task phase.

The predicted λ is used in two complementary ways. First, it is embedded as z_λ and injected into the action query, global memory, and AdaLN conditioning pathway. This allows the Transformer velocity expert to condition its representation on the predicted motion scale. Second, λ contributes directly to the residual step-scaling factor

$$\delta = \lambda \cdot \mathrm{Softplus}\left(\mathrm{MLP}_\delta([z_t, z_\lambda])\right),$$

which scales residual velocity-field updates in each Transformer block. Therefore, λ is not only a latent conditioning token; it also acts as an explicit action-scale modulator inside the velocity model.

During MP-CFM training, the same predicted λ is shared across all sampled flow-time points from the same demonstrated trajectory, while the action state and target velocity vary across sampled times. This exposes the magnitude predictor to multiple phase-dependent velocity targets along one demonstrated action path. As a result, the modulation pathway receives gradients from different motion phases, such as large reaching motions, contact-rich transitions, and fine terminal adjustments. This design encourages λ to encode observation-conditioned action-scale information useful for coarse-to-fine streaming control.

B.4 Shared Streaming Interface

Although the U-Net and DiT-style versions use different velocity backbones, they share the same ControlFlow interface:

$$v_\theta(a, t \mid h, \lambda).$$

Both versions generate actions through the same action-space integration rule:

$$a_{t+\Delta t} = a_t + v_\theta(a_t, t \mid h, \lambda)\Delta t.$$

The difference lies in how the observation context and modulation variable are injected into the velocity model. The U-Net version uses FiLM-style residual scaling inside convolutional residual blocks. The DiT-style version uses CLIP token conditioning, AdaLN-Zero modulation, and explicit frequency-scaled residual updates inside Transformer blocks.

This shared interface allows ControlFlow to support lightweight U-Net policies for controlled manipulation experiments and CLIP-conditioned DiT-style policies for language-conditioned multi-task settings such as LIBERO, without changing the streaming action-generation procedure.

C Efficiency Analysis

C.1 Parameter Efficiency

Table 5 compares the number of trainable parameters across different policies under state-based and image-based observation settings. ControlFlow introduces only a marginal increase in parameter count compared to baseline methods, as the adaptive modulation mechanism is implemented using a lightweight predictor without altering the backbone architecture.

C.2 Inference Efficiency

We evaluate inference-time efficiency by measuring the total execution time required to complete a full task rollout. Table 6 reports wall-clock inference time for ControlFlow and Diffusion Policy (DP) under identical evaluation settings.

Table 5: Comparison of model parameter counts under different observation modalities.

Policy	State Input	Image Input
Diffusion Policy (DP)	65M	72M
Streaming Flow Policy (SFP)	65M	72M
ControlFlow (ours)	67M	73.5M

Table 6: Inference time comparison between ControlFlow and Diffusion Policy.

Task	ControlFlow	DP
Push-T (State, 200 steps)	1.3 s	6.2 s
Can (State, 400 steps)	2.7 s	35.2 s
Square (State, 400 steps)	2.6 s	35.0 s

In addition to faster per-step inference, ControlFlow frequently completes tasks earlier than the predefined rollout horizon. In Can and Square, the policy often reaches successful termination after approximately 30% fewer execution steps compared to the maximum horizon. This early termination further reduces total inference time in practice and highlights the benefit of adaptive action magnitude control for efficient task execution.

C.3 LIBERO Inference Latency

We further analyze inference latency in the LIBERO language-conditioned setting. Since ControlFlow-T performs streaming execution with action chunking, we report two complementary latency measures. The first is *chunk-refresh latency*, defined as the wall-clock time required by one policy query to generate a new action chunk. The second is *amortized per-step policy latency*, defined as the average `select_action` latency over all executed control steps in an episode.

ControlFlow-T generates an 8-action chunk whenever the action queue is refreshed. At a refresh step, the policy performs one velocity-field integration pass, generates the full action chunk, and immediately returns the first action. The remaining actions are cached and returned from the queue in subsequent control steps. Thus, the policy incurs higher latency only at chunk-refresh steps, while intermediate steps retrieve cached actions with low overhead.

In our LIBERO evaluation, ControlFlow-T achieves an average chunk-refresh latency of approximately 75 ms for generating an 8-action chunk, and an amortized per-step policy latency of approximately 9 ms. The corresponding action-generation throughput at the chunk level is

$$\frac{8}{0.075} \approx 106.7$$

actions per second. This throughput measures policy-side action generation rather than the physical control frequency of the simulator or robot.

For comparison, OpenVLA reports approximately 240 ms query-level latency on LIBERO, while OpenVLA-OFT reports approximately 73 ms after optimized fine-tuning, parallel decoding, and action chunking [27, 28]. Under a query-level action-generation convention, ControlFlow-T is substantially faster than the original OpenVLA and close to the reported OpenVLA-OFT latency, while generating an 8-action chunk per refresh.

The key distinction is that ControlFlow-T also preserves streaming execution. For VLA baselines, query-level latency measures the time required to produce an action output or action chunk from a visual-language input. For ControlFlow-T, chunk-refresh latency provides the closest query-level comparison, whereas amortized per-step latency reflects the effective online cost after accounting for cached actions. Thus, ControlFlow-T provides a practical latency profile for language-conditioned manipulation: competitive chunk-level action generation together with low amortized per-step latency during streaming execution.

Table 7: Default training configurations for state-based and image-based models.

Parameter	**State-based**	**Image-based**
Optimizer	AdamW	AdamW
Adam β_1	0.9	0.9
Adam β_2	0.999	0.999
Adam ϵ	1×10^{-8}	1×10^{-8}
Learning rate	1×10^{-4}	2×10^{-4}
Weight decay	1×10^{-6}	1×10^{-6}
LR scheduler	Cosine	Cosine
Warmup steps	500	500
Batch size	256	1024
Gradient clipping	None	1.0
Mixed precision	FP16 (AMP)	FP16 (AMP)
EMA enabled	Yes	Yes
EMA decay	0.75	0.75
Image encoder output dim	–	512
Prediction horizon T_{pred}	16	16
Observation horizon T_{obs}	2	2
Action horizon T_{act}	8	8
CFM noise scale σ_0	0.4	0.4
CFM gain k	10	10

D Training Configuration

All training and evaluation experiments are conducted on a single NVIDIA A6000 GPU using PyTorch with CUDA acceleration. Table 7 summarizes the default configurations used for the U-Net-based state and image experiments on Push-T and RoboMimic. All models are optimized using AdamW, where β_1, β_2, and ϵ control the first- and second-moment estimation and numerical stability. A cosine learning rate schedule with linear warmup is employed to stabilize optimization during the early stage of training.

The horizons T_{obs}, T_{act}, and T_{pred} specify the lengths of observation context, executed action segment, and predicted action trajectory, respectively. In the conditional flow matching objective, the Gaussian noise scale σ_0 controls the magnitude of training perturbations, while the gain parameter k adjusts the strength of the trajectory-induced target velocity field. An exponential moving average (EMA) of model parameters is maintained during training to improve optimization stability and inference robustness. For image-based U-Net models, a ResNet-based image encoder is used to extract visual representations from RGB observations.

For the LIBERO experiments, we use ControlFlow-T, implemented as `streaming_flow_v4` in the LeRobot framework. This variant uses a DiT-style Transformer velocity expert with token-level CLIP image-text conditioning. Image observations are encoded by a frozen CLIP ViT-B/16 vision encoder, and language instructions are encoded by a frozen CLIP text encoder. Both pooled CLIP features and token-level representations are used: pooled features form the global conditioning vector for magnitude prediction and global modulation, while image patch tokens and text tokens serve as cross-attention memory for the DiT-style action expert. The observation-conditioned frequency-like variable is denoted as λ in the paper. The DiT-style model configuration and LIBERO training settings are summarized in Table 8. Mixed-precision training with automatic loss scaling is applied in all experiments to reduce memory usage and accelerate training.

Table 8: Training configuration for ControlFlow-T on LIBERO using a DiT-style CLIP-conditioned action expert.

Parameter	**Value**
Policy type	`streaming_flow_v4`
Benchmark	LIBERO
Task suites	Spatial, Object, Goal, Long
Dataset	`HuggingFaceVLA/libero`
Control mode	Relative
Observation size	256×256
Optimizer	AdamW
Batch size	128
Training steps	100k
Mixed precision	AMP
Number of workers	8
Video backend	`torchcodec`
Chunk size	16
Observation steps T_{obs}	2
Executed action steps T_{act}	8
MP-CFM supervision points N	8
CFM noise scale σ_0	0.4
CFM gain k	10.0
CLIP image conditioning	Yes
CLIP text conditioning	Yes
CLIP backbone	ViT-B/16
Frozen CLIP encoders	Yes
Image crop size	224×224
Random crop during training	Yes
Image feature dim	512
Text projection dim	256
Separate RGB encoder per camera	No
Embedding dim	256
Transformer hidden dim	768
Transformer layers	8
Transformer heads	12
Transformer FFN dim	3072
Transformer dropout	0.1

E Additional Discussion on Multi-Point CFM

Why multi-point supervision improves streaming stability. We provide an intuitive error-propagation analysis for the multi-point CFM objective. For a demonstrated trajectory $\xi(t)$, the trajectory-induced target velocity field is

$$v_\xi(a, t) = \dot{\xi}(t) - k(a - \xi(t)),$$

where $k > 0$ is a stabilizing gain. Let $\hat{a}(t)$ denote the action trajectory generated by the learned velocity field under the same observation context, and define the rollout error

$$e(t) = \hat{a}(t) - \xi(t).$$

Since $\dot{\hat{a}}(t) = v_\theta(\hat{a}(t), t \mid h, \lambda)$, we have

$$\dot{e}(t) = v_\theta(\hat{a}(t), t \mid h, \lambda) - \dot{\xi}(t).$$

Adding and subtracting $v_\xi(\hat{a}(t), t)$ gives

$$\dot{e}(t) = -ke(t) + \epsilon_\theta(t),$$

where

$$\epsilon_\theta(t) = v_\theta(\hat{a}(t), t \mid h, \lambda) - v_\xi(\hat{a}(t), t)$$

is the velocity approximation error evaluated along the generated trajectory. By the variation-of-constants formula,

$$e(t) = e^{-kt}e(0) + \int_0^t e^{-k(t-s)}\epsilon_\theta(s)\,ds.$$

Taking norms yields

$$\|e(t)\| \le e^{-kt}\|e(0)\| + \int_0^t e^{-k(t-s)}\|\epsilon_\theta(s)\|\,ds.$$

If $\|\epsilon_\theta(s)\| \le \bar{\epsilon}$ over the integration interval, then

$$\|e(t)\| \le e^{-kt}\|e(0)\| + \frac{1-e^{-kt}}{k}\bar{\epsilon}.$$

This bound does not claim a formal stability guarantee for the learned closed-loop policy. Instead, it shows that rollout deviation is directly controlled by the velocity approximation error accumulated along the integration path. Reducing velocity mismatch at more flow-time locations can therefore improve streaming rollout consistency.

MP-CFM improves this training signal by supervising multiple flow-time locations from the same demonstrated path:

$$t_1, \ldots, t_N \sim \mathcal{U}[0,1].$$

For each sampled time t_j, an action state $a_j \sim p_\xi(a \mid t_j)$ is sampled near the demonstrated trajectory, and the model minimizes

$$\frac{1}{N}\sum_{j=1}^{N} \|v_\theta(a_j, t_j \mid h, \lambda) - v_\xi(a_j, t_j)\|_2^2 .$$

Compared with single-time CFM, this provides denser coverage of the integration interval and encourages lower velocity mismatch across multiple phases of the same predicted action path. This is useful for streaming generation because local velocity errors can propagate through sequential integration and affect later action states.

Monte Carlo view. MP-CFM can also be viewed as a lower-variance estimator of a trajectory-wise velocity-matching objective. For a fixed demonstration (h, ξ), define the pointwise velocity-matching loss

$$\ell(t,a) = \frac{1}{2}\,\|v_\theta(a, t \mid h, \lambda) - v_\xi(a,t)\|_2^2 , \qquad a \sim p_\xi(a \mid t).$$

The trajectory-wise objective for this demonstration is

$$\mathcal{L}_\xi = \mathbb{E}_{t\sim\mathcal{U}[0,1]}\mathbb{E}_{a\sim p_\xi(a|t)}\left[\ell(t,a)\right].$$

Using N independently sampled pairs (t_j, a_j) conditional on the same demonstration gives

$$\widehat{\mathcal{L}}_\xi^{(N)} = \frac{1}{N}\sum_{j=1}^{N}\ell(t_j, a_j).$$

This estimator is unbiased:

$$\mathbb{E}\left[\widehat{\mathcal{L}}_\xi^{(N)} \mid h, \xi\right] = \mathcal{L}_\xi.$$

Under conditional independent sampling,

$$\mathrm{Var}\left[\widehat{\mathcal{L}}_\xi^{(N)} \mid h, \xi\right] = \frac{1}{N}\mathrm{Var}\left[\ell(t,a) \mid h, \xi\right].$$

For the full dataset objective, the total variance additionally includes variation across demonstrations:

$$\mathrm{Var}\left[\widehat{\mathcal{L}}_\xi^{(N)}\right] = \mathrm{Var}_{(h,\xi)}\left[\mathcal{L}_\xi\right] + \frac{1}{N}\mathbb{E}_{(h,\xi)}\left[\mathrm{Var}\left[\ell(t,a) \mid h, \xi\right]\right].$$

Thus, increasing N reduces the within-trajectory sampling variance while leaving the inference procedure unchanged.

Learning signal for the modulation variable λ. MP-CFM also provides a denser learning signal for the modulation variable λ. The same observation-conditioned value $\lambda = g_\phi(h)$ is shared across all sampled flow-time points from the same demonstration, while the action states and target velocities vary with t_j. For each sampled point, define the velocity residual

$$r_j = v_\theta(a_j, t_j \mid h, \lambda) - v_\xi(a_j, t_j),$$

with pointwise loss

$$\mathcal{L}_j = \frac{1}{2}\|r_j\|_2^2.$$

Since λ modulates the learned velocity field, the gradient received by the modulation predictor is

$$\frac{\partial \mathcal{L}_{\text{MP-CFM}}}{\partial \phi} = \frac{1}{N}\sum_{j=1}^{N} r_j^\top \frac{\partial v_\theta(a_j, t_j \mid h, \lambda)}{\partial \lambda}\frac{\partial \lambda}{\partial \phi}.$$

This expression shows that the modulation predictor is updated according to how changes in λ affect the velocity residual. Although no explicit motion-scale labels are provided, velocity-scale mismatch can produce gradients through the sensitivity term

$$\frac{\partial v_\theta(a_j, t_j \mid h, \lambda)}{\partial \lambda}.$$

With a single sampled time, the update to λ reflects one local velocity target. With N sampled times, the modulation predictor aggregates residual information from multiple flow-time locations:

$$\frac{1}{N}\sum_{j=1}^{N} r_j^\top \frac{\partial v_\theta(a_j, t_j \mid h, \lambda)}{\partial \lambda}.$$

This exposes λ to a broader range of target velocity magnitudes along the same predicted action path, such as larger updates during reaching phases and smaller updates during contact-rich or terminal alignment phases.

This analysis does not require λ to be a manually identifiable physical quantity. Rather, λ acts as an observation-conditioned modulation variable shaped by the CFM objective. Because the architecture injects λ through positive residual scaling, explicit step scaling, and gated shift-scale modulation, changes in λ primarily affect the effective magnitude of the velocity-field computation rather than directly specifying an action direction. Consequently, the learned λ is encouraged to encode the action-update scale required by the current observation context and task phase.

F Environment and Benchmark Details

RoboMimic. RoboMimic [21] is a standardized benchmark for robotic manipulation based on large-scale human and robot demonstration datasets. It is widely used for evaluating imitation learning and offline reinforcement learning methods under both state-based and visual observation settings. The benchmark includes manipulation tasks with diverse horizon lengths, contact dynamics, and control requirements, ranging from short-horizon object manipulation to precision and longer-horizon tasks.

In our experiments, we evaluate the U-Net-based ControlFlow model on five RoboMimic tasks: `Can`, `Lift`, `Square`, `ToolHang`, and `Transport`. `Can` and `Lift` evaluate basic object manipulation and lifting behaviors, while `Square` requires precise object placement. `ToolHang` further stresses fine-grained manipulation and tool alignment, and `Transport` evaluates longer-horizon object transfer. All RoboMimic environments are executed in the MuJoCo simulator [31] with standard benchmark configurations. For state-based experiments, policies receive low-dimensional proprioceptive and object-state observations as provided by the benchmark. For image-based experiments, policies operate on RGB observations rendered from fixed camera viewpoints. Unless otherwise specified, environment configurations, dataset splits, and evaluation settings follow those used in Diffusion Policy and related baseline implementations.

Push-T. Push-T [20] is a contact-rich planar manipulation task in which an agent pushes a T-shaped block toward a fixed target region using a circular end-effector. The task randomizes the initial positions and orientations of both the block and the end-effector at the beginning of each episode. Successful execution requires stable contact control and precise object alignment, making the task sensitive to action magnitude, control smoothness, and rollout stability.

We evaluate the U-Net-based ControlFlow model on Push-T under both state-based and image-based observations. In the image-based setting, the policy receives RGB observations from a top-down camera view. In the state-based setting, the policy receives a low-dimensional representation consisting of 2D keypoints derived from the ground-truth pose of the T-shaped block. During evaluation, performance is measured using the convergence-area metric introduced in the original Push-T benchmark, which quantifies the overlap between the pushed object and the target region rather than using a binary success signal. This metric is useful for evaluating whether a policy can perform fine-grained terminal adjustments near the target.

LIBERO. LIBERO [22] is a language-conditioned manipulation benchmark designed for studying multi-task and lifelong robot learning. It contains multiple task suites with different forms of variation, including spatial configurations, object identities, goal specifications, and longer-horizon task compositions.

We use LIBERO to evaluate ControlFlow-T, the DiT-style CLIP-conditioned variant of ControlFlow, in the LeRobot framework. In this setting, the policy receives visual observations and language task descriptions. Image observations are encoded using a CLIP visual encoder, while task instructions are encoded using a CLIP text encoder. The resulting visual, language, and proprioceptive features condition the DiT-style action expert.

Unlike the U-Net-based Push-T and RoboMimic experiments, the LIBERO experiments evaluate the scalability of the ControlFlow formulation under language-conditioned multi-task imitation learning. The streaming flow formulation and multi-point CFM objective remain unchanged; only the velocity-model backbone and conditioning interface are replaced by the DiT-style CLIP-conditioned version.

G Additional Analysis of Adaptive Modulation Factor λ

G.1 Temporal Evolution of Adaptive Lambda λ

Fig. 3 visualizes the temporal evolution of the predicted modulation factor λ during a representative Push-T rollout under image-based observations. At early stages, the predicted λ remains relatively large, supporting coarse, high-magnitude actions that quickly move the end-effector toward the object. During later stages, λ decreases as the object approaches the target region, allowing the policy to perform fine-grained corrections and improve target coverage without overshooting.

This behavior is consistent with the coarse-to-fine structure of the task. Large updates are useful during long-range motion, while smaller updates become preferable near contact-rich or terminal alignment phases. These results indicate that ControlFlow learns to adjust action magnitude according to the current task phase, rather than relying on a single fixed action scale.

G.2 Fixed versus Adaptive Lambda λ

We evaluate the effect of fixing λ versus dynamically predicting it from observations on the Push-T task with image-based inputs. Fig. 4 compares the average task score achieved under different fixed values of λ and the proposed adaptive strategy.

When λ is fixed, performance exhibits strong sensitivity to its value. Small values lead to insufficient action magnitude and poor task progress, while overly large values result in unstable behavior and degraded final performance. Although an intermediate fixed value can perform reasonably, no single

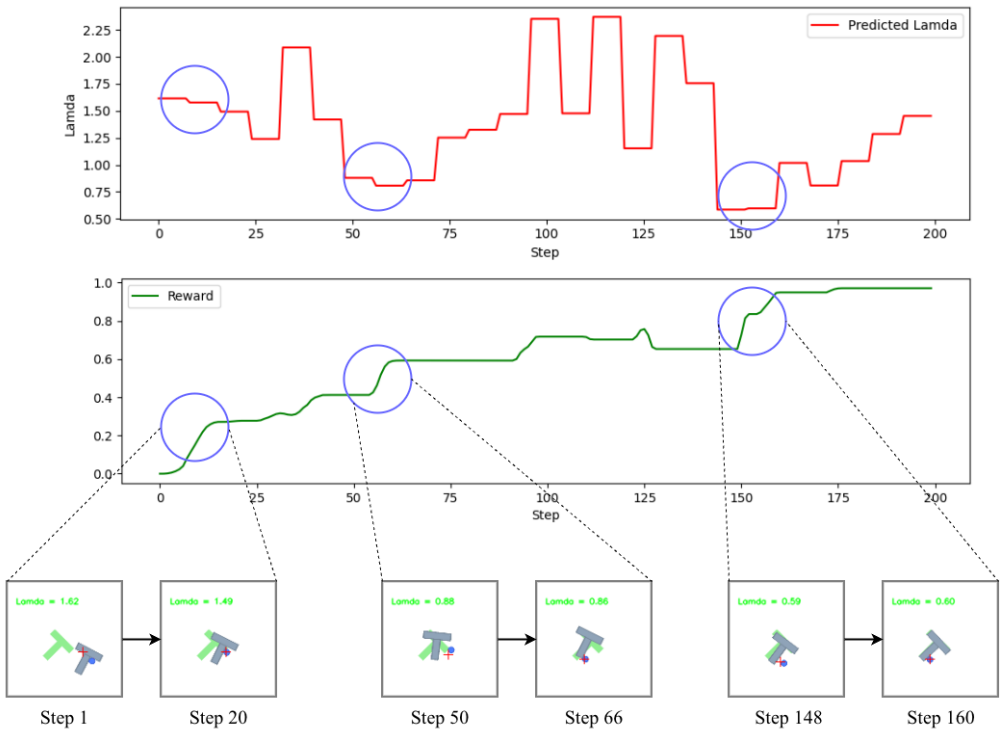


Figure 3: Temporal evolution of the predicted modulation factor λ and task reward during a representative Push-T rollout under image-based observations. **Top:** Predicted λ over execution steps. **Middle:** Corresponding task reward, computed based on the area of overlap between the pushed object and the target region. **Bottom:** Selected execution steps illustrating the interaction between action magnitude and task progress.

fixed λ is consistently optimal across executions. In contrast, the adaptive strategy achieves the highest average score by allowing λ to vary according to the current observation and task phase.

G.3 Reward-Binned Lambda Analysis

We further analyze the adaptive behavior of λ under different task outcomes. For this analysis, we collect predictions from the last 10 training checkpoints and group rollout trajectories according to achieved reward. For each reward interval, we compute the average predicted value of λ, as shown in Fig. 5.

The results reveal a reward-dependent modulation pattern. In high-reward regimes, λ tends to be smaller, indicating reduced action magnitude near task completion. In lower-reward regimes, λ is larger, reflecting the need for more aggressive action updates during earlier or less certain stages of execution.

H Action Horizon Analysis

We analyze the effect of action chunk horizon on policy performance for Push-T and RoboMimic Square, as shown in Fig. 6 and Fig. 7. Consistent with prior observations in diffusion-based and streaming policies, the action horizon exhibits a trade-off between responsiveness and temporal consistency. Shorter horizons provide faster feedback and stronger reactivity, while moderately longer horizons improve action smoothness and open-loop consistency. Performance typically peaks at an intermediate chunk size and degrades when the horizon becomes excessively long due to delayed

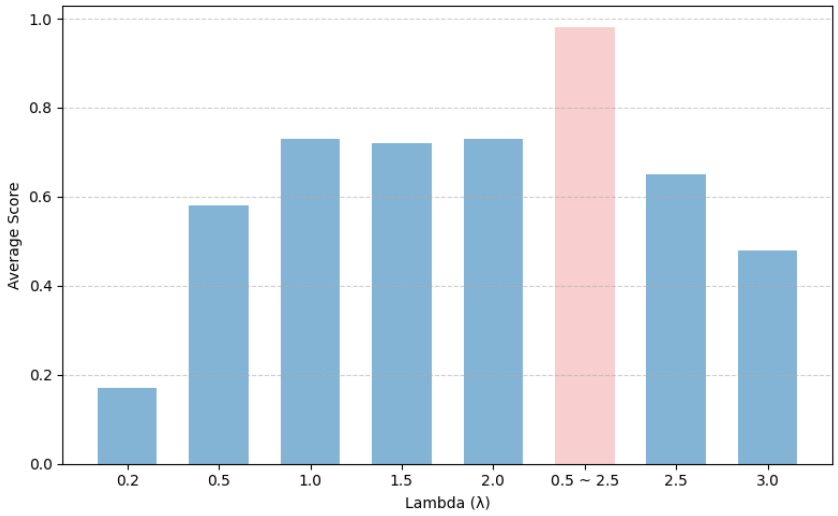


Figure 4: Comparison between fixed and adaptive λ on the Push-T task with image-based observations. Bars correspond to average scores achieved with different fixed values of λ, while the highlighted bar represents the adaptive strategy.

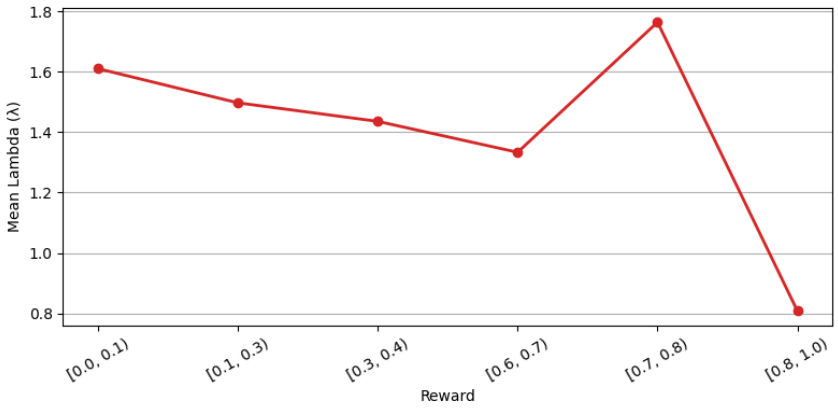


Figure 5: Average predicted modulation factor λ across different reward intervals on the Push-T task with image-based observations.

reaction to state changes, aligning with findings in Diffusion Policy [1] and Streaming Diffusion Policy [13].

ControlFlow maintains strong performance across a wide range of action horizons. Compared with baseline methods, its performance degradation under larger chunk sizes is reduced while remaining competitive at smaller horizons. We attribute this robustness to adaptive magnitude modulation, where the observation-conditioned factor λ adjusts the effective action magnitude according to task context and helps balance temporal consistency with receding-horizon responsiveness.

I Future Work and Limitation

Future Work. A promising direction is to extend ControlFlow from lightweight U-Net and DiT-style CLIP-conditioned instantiations to larger-scale Vision-Language-Action (VLA) models, such as π0.5 [32] and GR00T-N1 [33]. Although ControlFlow-T demonstrates compatibility with language-conditioned action experts, larger VLA systems provide richer multimodal context, stronger task priors, and broader cross-task generalization ability. Leveraging these representations to predict the

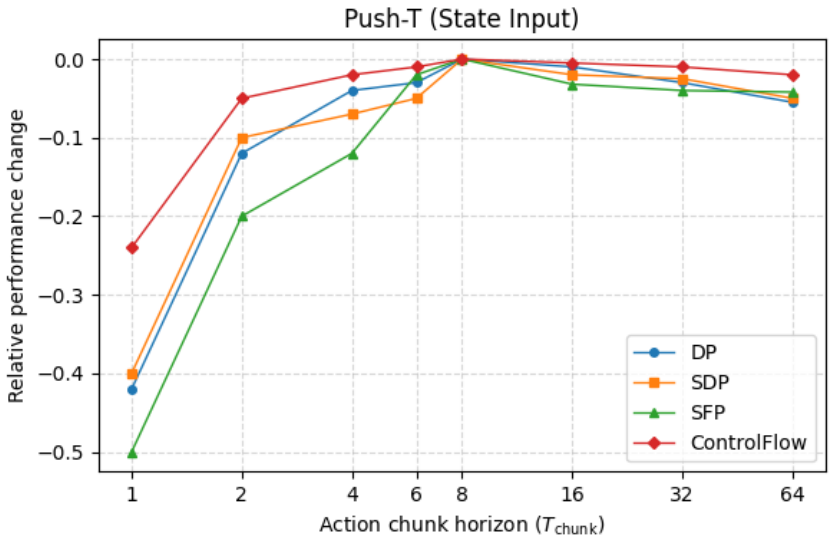


Figure 6: Effect of action chunk horizon on policy performance on Push-T under state-based observations. The x-axis shows the action chunk size T_{chunk} in log scale. The y-axis shows the relative decrease in performance compared to the best-performing chunk size.

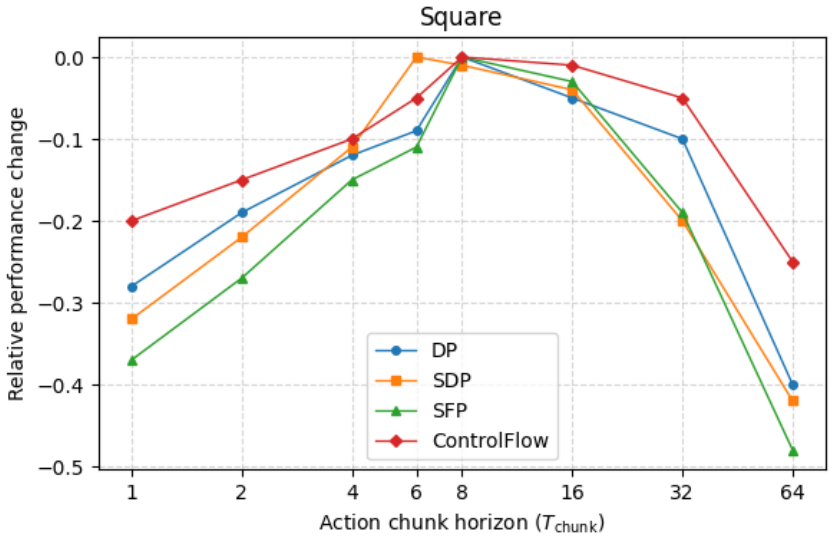


Figure 7: Effect of action chunk horizon on policy performance on RoboMimic Square under state-based observations. The axes follow the same definition as Fig. 6.

modulation variable λ could further improve phase-aware control in long-horizon and multi-skill manipulation.

Another direction is to integrate adaptive magnitude modulation into flow-based VLA decoders without disrupting their existing action-generation interfaces. Since ControlFlow modulates the velocity field through a lightweight observation-conditioned variable, it can be incorporated as an additional conditioning pathway rather than requiring a complete redesign of the policy backbone. Exploring how λ should be predicted from multimodal context, language instructions, and task progress while preserving low-latency streaming execution remains an important avenue for future work.

Limitation. ControlFlow focuses on adaptively modulating the magnitude of the velocity field from the current observation context. This improves the transition between coarse motion and fine-grained adjustment, but it does not by itself increase the expressiveness or coverage of the underlying action representation. Thus, performance remains dependent on the quality, resolution, and diversity of the demonstration data.

In highly dexterous or contact-sensitive tasks, adaptive magnitude control may reduce overshooting and improve rollout stability, but it cannot fully compensate for missing demonstrations, ambiguous visual observations, or insufficient contact-rich training data. Although the DiT-style variant extends ControlFlow to language-conditioned multi-task settings, scaling the method to foundation-level VLA systems requires further study of multimodal conditioning, cross-task transfer, and real-world deployment stability.